

计算机组成

单周期数据通路形式建模综合方法

(2025秋)

高小鹏

北京航空航天大学计算机学院

目录

- ▣ 设计方法学概述
- ▣ 单条指令的数据通路的建模方法
- ▣ 完整数据通路的综合方法

系统的观点

- 系统是相互作用的诸元素的综合体。系统的特点：
 - ◆ ①一个系统包含至少2个以上的要素
 - ◆ ②要素之间是存在连接关系的
 - ◆ ③要素的功能以及要素间的连接关系决定了系统的整体功能
- 如果把CPU视为系统，则寄存器堆、ALU、控制器等就是要素
- 宏观上，一个CPU的功能正确性取决于：
 - ◆ 各个功能部件的正确性
 - ◆ 功能部件间建立了正确的连接关系

问题

CPU被分解为寄存器堆、ALU等功能部件，遵循的是什么方法或原则？

※要素也可以被称为子系统

※要素、子系统、功能部件是含义相同的表述方式

计算机组成与实现

机制与策略

- 机制：按照功能最大无关性及组合最大灵活性为基本原则构造的功能部件的集合
- 策略：根据高层需求选取并控制功能部件子集执行相应的功能

	CPU	人	路口交通控制
高层需求	执行指令	走、跑、跳	各种通行需求
机制	程序计数器 寄存器堆 算术逻辑单元 指令存储器 数据存储器	四肢 躯干	各路口的红绿灯 各灯的定时器
策略	控制器	大脑	交通控制箱

※要素也可以被称为子系统

计算机组成与实现

机制与策略分离的设计方法学

- 降低系统复杂度。
 - ◆ 多数系统包含大量高度相似的功能需求，将其由相对独立的功能部件实现
 - ◆ 通过设计和修改策略部分就能很容易的实现一个个的具体需求了
- 提高系统灵活性
 - ◆ 假设功能部件是较为充分，则可以仅通过增加新的策略来实现某个新需求
- 提高系统扩展性
 - ◆ 当新需求无法通过现有机制与策略无法满足时，可以将其转换为相应的机制与策略，然后添加到已有的机制与策略集合中
 - ◆ 该设计模式对已有设计影响(如修改、重构、甚至推倒重建)可能是最小的
- 提高系统稳定性
 - ◆ 在机制与策略分离设计模式下，通常每个功能部件的规模都非常有限
 - ◆ 规模有限通常意味着复杂度低、出错概率低
 - ◆ 如此，最终集成起来的系统就会具有足够的可靠性

计算机组成与实现

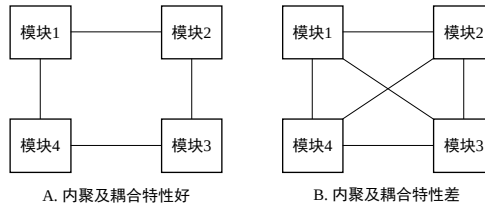
高内聚与低耦合

- 内聚：是指某个功能部件的内部各功能的相关性
- 耦合：是指功能部件间连接关系的强度
- 高内聚
 - ◆ 强调单一性，即功能部件或者其功能单一或者其多个功能高度相似
 - ◆ 遵循的是单一职责原则(Single Responsibility Principle)
- 低耦合
 - ◆ 强调功能部件的接口应该尽可能少，彼此间的连接关系尽可能的简单
 - ◆ 遵循的是最少知识原则(Least Knowledge Principle)或迪米特法则(Law of Demeter)
- 高内聚低耦合是更具可操作性的系统设计原则
 - ◆ 高内聚的设计结构几乎必然导致低耦合
 - ◆ 低耦合必然意味着高内聚

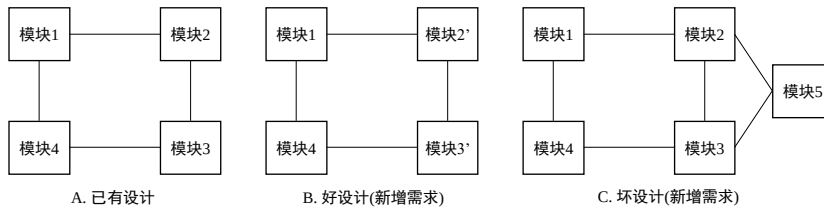
计算机组成与实现

高内聚与低耦合

- 示例1：方案A比方案B具有更低的耦合度，因此优于方案B



- 示例2：对于新需求，方案B仅仅修改了已有模块，而方案C则引入了新模块，因此方案B优于方案C

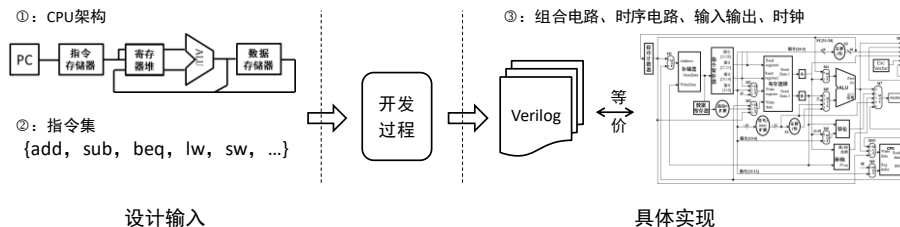


计算机组成与实现

形式建模综合方法概述

- CPU开发是什么？

- ◆ 以某种CPU架构（如单周期、多周期或流水线）为模型，面向给定的指令集，设计并构造出以Verilog形式表达的CPU具体实现



- CPU架构模型是什么？

- ◆ CPU架构模型：以寄存器堆、ALU等功能部件为基础，描述了各功能部件的基本接口特性以及相互间的基本连接关系
- ◆ CPU架构模型关注两点
 - 功能部件的外特性，即功能部件的某个具体功能与控制的对应关系
 - 功能部件间传递信息的基本依赖关系

计算机组成与实现

CPU开发基本过程

- 1) 构造功能部件。构造一组必要的通路功能部件。对于每个功能部件，都需要定义其功能及接口信号，并用HDL（Hardware Description Language，硬件描述语言）描述其内部具体的电路行为或结构。
- 2) 构造指令级别的数据通路与控制信号取值。根据指令操作语义构造指令级别的数据通路及相关功能部件的控制信号取值。
- 3) 综合数据通路。当所有指令级别的数据通路构造完毕后，将其综合为完整数据通路。
- 4) 综合控制器。当全部指令对应的控制信号取值均建立后，合并所有的控制信号取值形成控制信号矩阵，并生成相应的控制信号表达式。
- 5) 生成工程项目。将功能第2步和第3步的结果翻译为HDL语言，并在顶层工程文件中组装数据通路及控制器。

计算机组成与实现

形式建模综合方法概述

- 形式建模综合方法的要点是什么？
 - ◆ 1) 开发过程是基于模型的
 - 这里的模型就是所谓的CPU架构，如单周期、多周期、流水线
 - ◆ 2) 开发过程被显式的分为设计和实现两个环节
 - 设计：是指建模每条指令的数据通路和控制信号
 - 实现：是指将设计结果用Verilog等语言表达
 - ◆ 3) 基于“系统-子系统”视角的建模层次
 - CPU被视为系统，各功能部件被视为子系统
 - ◆ 4) 指令级别独立建模
 - 独立分析每条指令操作语义，推演其执行过程中的数据流信息和控制流信息
 - 独立构造每条指令对应的数据通路以及相关功能部件的控制信号取值
 - ◆ 5) 一次性的系统级综合
 - 将所有分离的数据通路及控制信号取值高效合成为完整的数据通路和控制器

计算机组成与实现

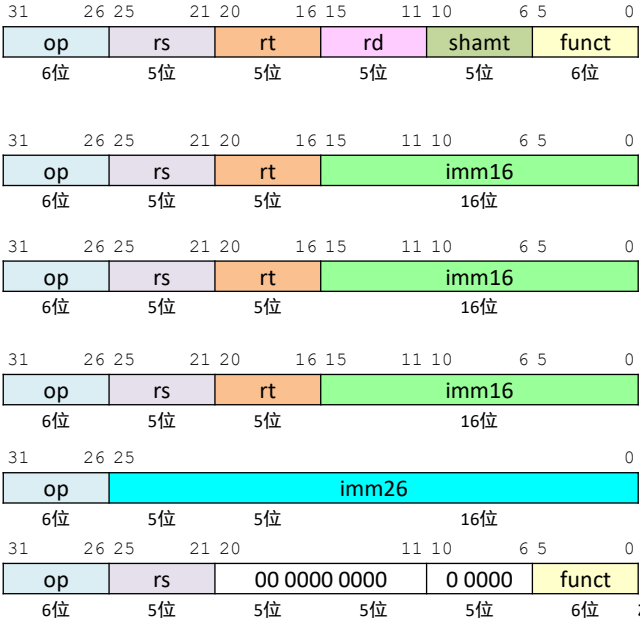
形式建模综合方法概述

- 指令级别独立建模是什么？
 - ◆ 1) 根据指令的RTL，梳理和总结出数据通路的设计需求
 - ◆ 2) 选择恰当的数据通路功能部件
 - ◆ 3) 建立功能部件间的正确连接关系
 - ◆ 4) 根据指令的RTL，选择功能部件应执行的功能，反推控制信号取值

计算机组成与实现

MIPS-C0指令集

- 加减法指令
 - ◆ addu rd,rs,rt
 - ◆ subu rd,rs,rt
- 立即数指令
 - ◆ ori rt,rs,imm16
- 存储访问指令
 - ◆ lw rt,rs,imm16
 - ◆ sw rt,rs,imm16
- 分支指令
 - ◆ beq rs,rt,imm16
- 函数调用指令
 - ◆ jal target
- 函数返回指令
 - ◆ jr rs



机组成与实现

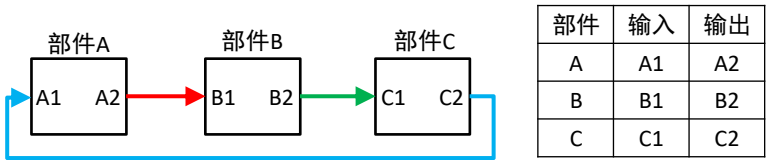
目录

- 设计方法学概述
- 单条指令的数据通路的建模方法
 - Addu
- 完整数据通路的综合方法

计算机组成与实现

建模的基本表示方式

- 数据通路本质上是一个连接关系的集合
 - 集合的元素：相关部件的输入输出信号之间的连接关系
- 示例：3个部件的连接关系



- 连接关系集合：{<C.C2,A.A1>, <A.A2,B.B1>, <B.B2,C.C1>}
- 连接关系<X.m,Y.n>：部件X的输出信号m，连接至部件Y的输入信号n
- 连接关系表格：该方式表示连接关系集合，在布局上更易于对应图形方式

A	B	C	----- 部件
A1	B1	C1	----- 部件的输入信号
C.C2	A.A2	B.B2	----- 部件及其输出信号

计算机组成与实现

建模的基本推理过程

- 基本事实1：当CPU基本模型确定后，意味着指令的执行路径总体是确定的
 - 执行路径也可以认为是信息流路径
 - 执行路径大体包括：取指、译码/读操作数、计算、访存、回写
- 基本事实2：构成指令执行路径的各分段之间存在着明确的依赖关系
 - 示例1：指令执行的前提是必须先读取指令；而读取指令就必然是用PC驱动IM的地址线，然后IM输出的就是当前要执行的指令
 - 示例2：读取DM的前提是必须先计算出地址；而地址是通过加法运算完成的，因此ALU执行加法就是非常合理的

计算机组成与实现

建模的推理方法：倒推法

- 由于指令执行存在强逻辑依赖关系，因此建模指令基本的数据通路时既可采用正向推理方法，也可采用反向推理法，即倒推法
 - 反向推理结束后，将推理顺序倒置即得到正向的推理顺序

环节	步骤	语义	连接关系	功能部件	控制信号取值
-1	A	每步的具体操作	<X.m, Y.n>	F	信号名:取值
-2	B				
-3	C				



环节	步骤	语义	连接关系	功能部件	控制信号取值
1	C				
2	B				
3	A	每步的具体操作	<X.m, Y.n>	F	信号名:取值

计算机组成与实现

数据通路表

- 为了直观展示及易于综合，每条指令推理结束后，用数据通路表记录该指令的所有连接关系
- 示例：包含PC、NPC、IM、RF、ALU、DM的数据通路表
 - 第1行：为功能部件名
 - 第2行：该功能部件的各个输入信号
 - 第3行：与该输入信号连接的某个功能部件的输出信号

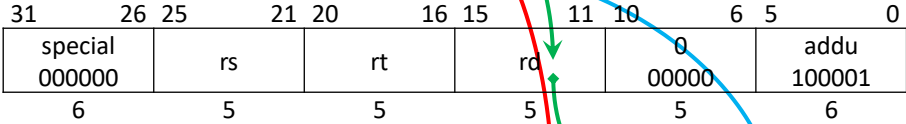
指令	NPC	PC	IM	RF				ALU		DM	
	PC	DI	A	A1	A2	A3	WD	A	B	A	WD

计算机组成与实现

Addu建模

$$R[rd] \leftarrow R[rs] + R[rt]$$

- 最后环节：必定是将加法运算的计算结果写入寄存器堆
- 写入寄存器堆涉及3个要素，即写入编号、写入数据、写使能
 - 基本事实：ALU具有加法功能
 - 推理1：写入编号来自指令的rd域
 - 推理2：写入数据来自ALU
 - 推理3：写使能需要有效，即为1



环节	步骤	语义	连接关系	功能部件	控制信号取值
-1	回写	计算结果回写至rd寄存器	$\langle \text{ALU.C}, \text{RF.WD} \rangle$ $\langle \text{IM.D}[15:11], \text{RF.A3} \rangle$	RF	$\text{RFWr}: 1$

计算机组成与实现

Addu建模

$R[rd] \leftarrow R[rs] + R[rt]$

- 将连接关系用表格表示
 - 控制信号取值暂时不表示，后续会在控制器设计中使用

环节	步骤	语义	连接关系	功能部件	控制信号取值
-1	回写	计算结果回写至rd寄存器	$\langle \text{ALU.C}, \text{RF.WD} \rangle$ $\langle \text{IM.D}[15:11], \text{RF.A3} \rangle$	RF	$\text{RFWr}:1$

指令	NPC	PC	IM	RF				ALU		DM	
				A1	A2	A3	WD	A	B	A	WD
addu						$\text{IM.D}[15:11]$	ALU.C				

计算机组成与实现

Addu建模

$R[rd] \leftarrow R[rs] + R[rt]$

- 2环节：既然写入数据来自ALU，因此需要由ALU完成加法计算
- ALU运算涉及3个要素，即第1个操作数、第2个操作数、ALU操作
 - 推理1：第1个操作数是从RF的RD1输出
 - 推理2：第2个操作数是从RF的RD2输出
 - 推理3：ALU控制信号取值为加法编码

环节	步骤	语义	连接关系	功能部件	控制信号取值
-1	回写	计算结果回写至rd寄存器	$\langle \text{ALU.C}, \text{RF.WD} \rangle$ $\langle \text{IM.D}[15:11], \text{RF.A3} \rangle$	RF	$\text{RFWr}:1$
-2	执行	ALU执行加法	$\langle \text{RF.RD1}, \text{ALU.A} \rangle$ $\langle \text{RF.RD2}, \text{ALU.B} \rangle$	ALU	$\text{ALUOp}:\text{ADD}$

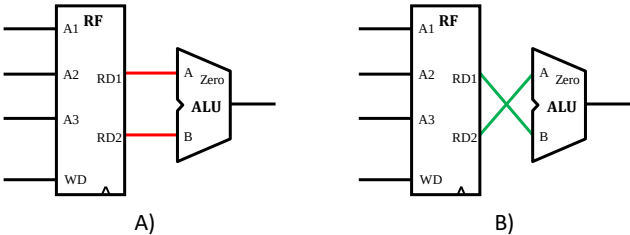
这是具有可读性的宏定义

指令	NPC	PC	IM	RF				ALU		DM	
	PC	NPC	A	A1	A2	A3	WD	A	B	A	WD
addu				$\text{IM.D}[25:21]$	$\text{IM.D}[20:16]$	$\text{IM.D}[15:11]$	ALU.C	RF.RD1	RF.RD2		

计算机组成与实现

RF与ALU的连接关系

- RF有2个32位输出，RD1和RD2；ALU有2个输入，A和B
- 问题：所有指令的RTL都不定义两者的连接关系，那么ALU的A/B连接RF的RD1/RD2还是RD2/RD1？



解答

- 1) 对于任一指令而言，两种连接均可以
- 2) 推荐采用固定连接方案（习惯上采用方案A）

※※※

如果有些指令采用方案A，有些指令采用方案B，这不影响功能正确性。但是，今后综合完整数据通路时需要更多MUX。

计算机组成与实现

Addu建模

$$R[rd] \leftarrow R[rs] + R[rt]$$

- 3环节：为了支持ALU计算，RF需要读取2个操作数
- RF读取操作数涉及2个要素，即第1操作数编号、第2操作数编号
 - 推理1：第1操作数编号是指令的rs域（先假设RF的RD1输出对应rs）
 - 推理2：第2操作数编号是指令的rt域（先假设RF的RD2输出对应rt）

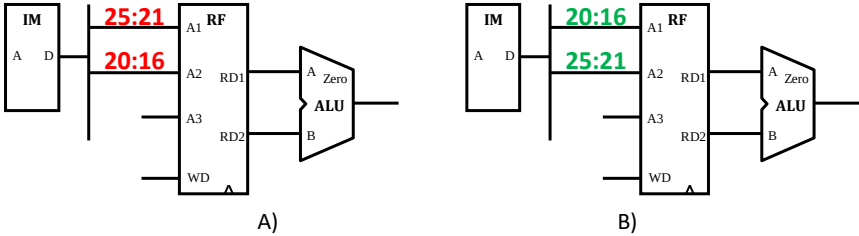
环节	步骤	语义		连接关系				功能部件	控制信号取值			
	31	26	25	21	20	16	15	11	10	6	5	0
	special 000000		rs		rt			rd		0 00000		addu 100001
	6		5		5			5		5		6
-3	读操作数		<IM.F[25:21], RF.A1> <IM.D[20:16], RF.A2>							RF		

指令	NPC	PC	IM	RF				ALU		DM	
	PC	NPC	A	A1	A2	A3	WD	A	B	A	WD
addu				IM.D[25:21]	IM.D[20:16]	IM.D[15:11]	ALU.C	RF.RD1	RF.RD2		

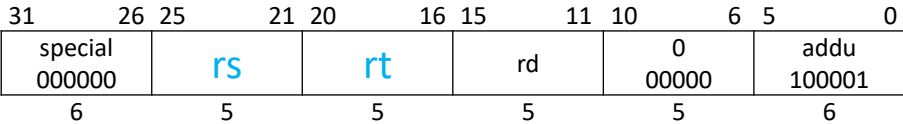
计算机组成与实现

IM与RF的连接关系

- RF有2个读寄存器编号，A1和A2
- 问题：RF的A1/A2对应rs/rt还是rt/rs？



- 解答
 - 1) 对于任一指令而言，两种连接均可以
 - 2) 推荐采用**固定**连接方案（为保持一致性，习惯上采用方案A）



计算机组成与实现

Addu建模

$R[rd] \leftarrow R[rs] + R[rt]$

- 4环节：必然是取指令
- 取指令涉及2个要素，即**读取IM**、**计算PC+4**、**更新PC**
 - 推理1：为了**读取IM**，需要PC驱动IM的地址线
 - 推理2：为了**计算PC+4**，需要PC输入至NPC
 - 推理3：为了**更新PC**，需要将NPC计算结果再写入PC

环节	步骤	语义	连接关系	功能部件	控制信号取值
-1	回写	计算结果回写至rd寄存器	$\langle \text{ALU.C}, \text{RF.WD} \rangle$ $\langle \text{IM.D}[15:11], \text{RF.A3} \rangle$	RF	RFWr:1
-2	执行	ALU执行加法	$\langle \text{RF.RD1}, \text{ALU.A} \rangle$ $\langle \text{RF.RD2}, \text{ALU.B} \rangle$	ALU	ALUOp:ADD
-3	读操作数		$\langle \text{IM.D}[25:21], \text{RF.A1} \rangle$ $\langle \text{IM.D}[20:16], \text{RF.A2} \rangle$	RF	
-4	读指令	读取指令 计算下一条指令地址 更新PC	$\langle \text{PC.DO}, \text{IM.A} \rangle$ $\langle \text{PC.DO}, \text{NPC.PC} \rangle$ $\langle \text{NPC.NPC}, \text{PC.DI} \rangle$	IM PC NPC	

指令	NPC	PC	IM	RF				ALU		DM	
	PC	DI	A	A1	A2	A3	WD	A	B	A	WD
addu	PC.DO	NPC.NPC	PC.DO	IM.D[25:21]	IM.D[20:16]	IM.D[15:11]	ALU.C	RF.RD1	RF.RD2		

计算机组成与实现

Addu建模

$R[rd] \leftarrow R[rs] + R[rt]$

将推理过程倒置后，得到正向的推理过程

环节	步骤	语义	连接关系	功能部件	控制信号取值
1	读指令	读取指令 计算下条指令地址 更新PC	$\langle PC.DO, IM.A \rangle$ $\langle PC.DO, NPC.PC \rangle$ $\langle NPC.NPC, PC.DI \rangle$	IM PC NPC	
2	读操作数		$\langle IM.D[25:21], RF.A1 \rangle$ $\langle IM.D[20:16], RF.A2 \rangle$	RF	
3	执行	ALU执行加法	$\langle RF.RD1, ALU.A \rangle$ $\langle RF.RD2, ALU.B \rangle$	ALU	ALUOp:ADD
4	回写	计算结果回写至rd寄存器	$\langle ALU.C, RF.WD \rangle$ $\langle IM.D[15:11], RF.A3 \rangle$	RF	RFWr:1

将推理得到的连接关系转换为表格表示

- 示例：红绿蓝三色对应环节1的3个连接关系

指令	NPC	PC	IM	RF				ALU		DM	
	PC	DI	A	A1	A2	A3	WD	A	B	A	WD
addu	PC.DO	NPC.NPC	PC.DO	IM.D[25:21]	IM.D[20:16]	IM.D[15:11]	ALU.C	RF.RD1	RF.RD2		

计算机组成与实现

目录

- 设计方法学概述
- 单条指令的数据通路的建模方法
 - Subu
- 完整数据通路的综合方法

计算机组成与实现

Subu建模

$R[rd] \leftarrow R[rs] - R[rt]$

▣ Subu与addu高度相似，区别仅仅在于ALU执行**减法运算**

环节	步骤	语义	连接关系	功能部件	控制信号取值
1	读指令	读取指令 计算下条指令地址 更新PC	<PC.DO, IM.A> <PC.DO, NPC.PC> <NPC.NPC, PC.DI>	IM PC NPC	
2	读操作数		<IM.D[25:21], RF.A1> <IM.D[20:16], RF.A2>	RF	
3	执行	ALU执行加法	<RF.RD1, ALU.A> <RF.RD2, ALU.B>	ALU	ALUOp: SUB
4	回写	计算结果回写至rd寄存器	<ALU.C, RF.WD> <IM.D[15:11], RF.A3>	RF	RFWr: 1

指令	NPC	PC	IM	RF				ALU		DM	
	PC	DI	A	A1	A2	A3	WD	A	B	A	WD
subu	PC.DO	NPC.NPC	PC.DO	IM.D[25:21]	IM.D[20:16]	IM.D[15:11]	ALU.C	RF.RD1	RF.RD2		

计算机组成与实现

目录

- ▣ 设计方法学概述
- ▣ **单条指令的数据通路的建模方法**
 - ◆ Ori
- ▣ 完整数据通路的综合方法

计算机组成与实现

Ori建模

$R[rt] \leftarrow R[rs] \text{ OR } \text{zero_ext}(\text{imm16})$

- zero_ext(): 这是一个新的计算需求
 - ◆ 原有的功能部件无法满足该需求
- EXT: 新增功能部件, 用于将16位数进行0扩展为32位数

信号名称	方向	描述
Imm16[15:0]	输入	16位输入。
Ext[31:0]	输出	32位0扩展结果。

HDL建模: Extender.v

```
module EXT( Imm16, Ext ) ;
    . . .
    assign Ext = {16{0}, Imm16} ;
end module
```

机组成与实现

Ori建模

$R[rt] \leftarrow R[rs] \text{ OR } \text{zero_ext}(\text{imm16})$

- Ori与addu/sub非常相似, 区别在于
 - ◆ 1) ALU的第2个操作数为EXT的扩展结果
 - ◆ 2) ALU执行OR运算
 - ◆ 3) 写入rt寄存器
 - ◆ 3)

环节	步骤	语义	连接关系	功能部件	控制信号取值
1	读指令	读取指令 计算下条指令地址 更新PC	$\langle PC.DO, IM.A \rangle$ $\langle PC.DO, NPC.PC \rangle$ $\langle NPC.NPC, PC.DI \rangle$	IM PC NPC	
2	读操作数		$\langle IM.D[25:21], RF.A1 \rangle$ $\langle IM.D[15:00], EXT.IMM16 \rangle$	RF EXT	
3	执行	ALU执行加法	$\langle RF.RD1, ALU.A \rangle$ $\langle EXT.Ext, ALU.B \rangle$	ALU	ALUOp:OR
4	回写	计算结果回写至rd寄存器	$\langle ALU.C, RF.WD \rangle$ $\langle IM.D[20:16], RF.A3 \rangle$	RF	RFWr:1

指令	NPC	PC	IM	RF				EXT	ALU		DM	
	PC	NPC	A	A1	A2	A3	WD	Imm16	A	B	A	WD
addu	PC.DO	NPC.NPC	PC.DO	IM.D[25:21]		IM.D[20:16]	ALU.C	IM.D[15:0]	RF.RD1	EXT.Ext		

计算机组成与实现

目录

- 设计方法学概述
- 单条指令的数据通路的建模方法
 - ◆ **Lw**
- 完整数据通路的综合方法

计算机组成与实现

Lw建模

$$R[rt] \leftarrow \text{MEM}[R[rs] + \text{sign_ext}(\text{imm16})]$$

- 新增DM功能部件
 - ◆ 由于不是写存储器操作，因此DM.WD无需连接
- 新增符号扩展功能
 - ◆ 0扩展与符号扩展，其输入位数、输出位数及基本目的相同
 - ◆ 根据“高内聚、低耦合”原则，由EXT同时实现两种扩展较为合理
 - 由于EXT同时支持两种扩展，因此必须增加**控制信号EXTOp**

信号名称	方向	描述
Imm[15:0]	输入	16位输入。
EXTOp	输入	扩展功能选择 0：符号扩展 1：无符号扩展
Ext[31:0]	输出	32位0扩展结果。

计算机组成与实现

Lw建模

$R[rt] \leftarrow MEM[R[rs] + sign_ext(imm16)]$

- 新增DM功能部件
 - 由于不是写存储器操作，因此DM.WD无需连接
- 新增符号扩展功能
 - 0扩展与符号扩展，其输入位数、输出位数及基本目的相同
 - 根据“高内聚、低耦合”原则，由EXT同时实现两种扩展较为合理
 - 由于EXT同时支持两种扩展，因此必须增加控制信号EXTOp

HDL建模：Extender.v

```
module EXT( Imm, F, Ext ) ;
    . . .
    assign Ext = EXTOp==`ZEXT ? {16{0}, Imm} :
                                   {16{Imm[15]}, Imm} ;
end module
```

书写表达式时，尽量多使用宏，增加可读性。

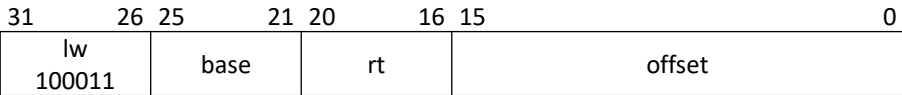
机组成与实现

Lw建模

$R[rt] \leftarrow MEM[R[rs] + sign_ext(imm16)]$

- ALU输入寄存器与扩展数，执行加法，结果输出至DM地址
 - 1) 地址计算与ori的计算过程类似（寄存器与扩展数运算）
 - 2) ALU具有执行加法的功能
- 为完成回写，DM的数据输出连接至RF的WD，且DM写使能为1

环节	步骤	语义	连接关系	功能部件	控制信号取值
1	读指令	读取指令 计算下条指令地址 更新PC	<PC.DO, IM.A> <PC.DO, NPC.PC> <NPC.NPC, PC.DI>	IM PC NPC	
2	读操作数		<IM.D[25:21], RF.A1> <IM.D[15:00], EXT.IMM16>	RF EXT	EXTOp:SEXT
3	执行	ALU执行加法	<RF.RD1, ALU.A> <EXT.Ext, ALU.B>	ALU	ALUOp:ADD
4	访存	读取DM	<ALU.C, DM.A>	DM	
4	回写	计算结果回写至rd寄存器	<DM.RD, RF.WD> <IM.D[20:16], RF.A3>	RF	RFWr:1



机组成与实现

Lw建模

$R[rt] \leftarrow \text{MEM}[R[rs] + \text{sign_ext}(\text{imm16})]$

环节	步骤	语义	连接关系	功能部件	控制信号取值
1	读指令	读取指令 计算下条指令地址 更新PC	<PC.DO, IM.A> <PC.DO, NPC.PC> <NPC.NPC, PC.DI>	IM PC NPC	
2	读操作数		<IM.D[25:21], RF.A1> <IM.D[15:00], EXT.IMM16>	RF EXT	EXTOp:SEXT
3	执行	ALU执行加法	<RF.RD1, ALU.A> <EXT.Ext, ALU.B>	ALU	ALUOp:ADD
4	访存	读取DM	<ALU.C, DM.A>	DM	
5	回写	计算结果回写至rd寄存器	<DM.RD, RF.WD> <IM.D[20:16], RF.A3>	RF	RFWr:1

指令	NPC	PC	IM	RF				EXT	ALU		DM	
	PC	NPC	A	A1	A2	A3	WD	Imm16	A	B	A	WD
lw	PC.PC	NPC.NPC	PC.PC	IM.D[25:21]		IM.D[20:16]	DM.RD	IM.D[15:0]	RF.RD1	EXT.Ext	ALU.C	

- 注意：由于EXT部件被修改了，因此凡是与该部件相关的指令均需被重新建模
 - 主要是建模需要考虑EXT的控制信号取值

计算机组成与实现

目录

- 设计方法学概述
- 单条指令的数据通路的建模方法
 - Sw
- 完整数据通路的综合方法

计算机组成与实现

Sw建模

$$\text{MEM}[\text{R}[\text{rs}] + \text{sign_ext}(\text{imm16})] \leftarrow \text{R}[\text{rt}]$$

- Sw与lw在地址计算方面完全一致
- Sw与lw不同点在于其不需要读DM，而是将rt寄存器写入DM
 - 即RF的RD2要写入DM

环节	步骤	语义	连接关系	功能部件	控制信号取值
1	读指令	读取指令 计算下条指令地址 更新PC	$\langle \text{PC.D0}, \text{IM.A} \rangle$ $\langle \text{PC.D0}, \text{NPC.PC} \rangle$ $\langle \text{NPC.NPC}, \text{PC.DI} \rangle$	IM PC NPC	
2	读操作数		$\langle \text{IM.D}[25:21], \text{RF.A1} \rangle$ $\langle \text{IM.D}[15:00], \text{EXT.IMM16} \rangle$	RF EXT	EXTOp:SEXT
3	执行	ALU执行加法	$\langle \text{RF.RD1}, \text{ALU.A} \rangle$ $\langle \text{EXT.Ext}, \text{ALU.B} \rangle$	ALU	ALUOp:ADD
4	访存	读取DM	$\langle \text{ALU.C}, \text{DM.A} \rangle$ $\langle \text{RF.RD2}, \text{DM.WD} \rangle$	DM	DMWR:1

指令	NPC	PC	IM	RF				EXT	ALU		DM	
	PC	NPC	A	A1	A2	A3	WD	Imm16	A	B	A	WD
lw	PC.PC	NPC.NPC	PC.PC	IM.D[25:21]				IM.D[15:0]	RF.RD1	EXT.Ext	ALU.C	RF.RD2

计算机组成与实现

目录

- 设计方法学概述
- 单条指令的数据通路的建模方法
 - Beq
- 完整数据通路的综合方法

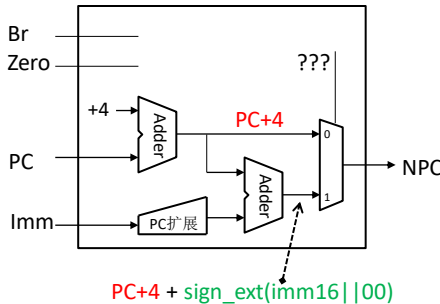
计算机组成与实现

Beq建模

本质上，beq涉及2大功能

- 功能1：根据比较的结果，计算PC。这属于NPC的功能范畴
- 功能2：寄存器比较。让ALU执行减法，然后把比较结果zero传递给NPC

对于NPC，现在需要知道当前指令是否是beq及zero的结果



```
if ( R[rs] == R[rt] )
    PC ← PC+4 + sign_ext(imm16||00)
else
    PC ← PC+4
```

信号名	方向	描述
PC[31:0]	I	32位输入
Imm[15:0]	I	16位立即数
Br	I	beq指令标志 1: 当前指令是beq 0: 当前指令不是beq
Zero	1	rs和rt相等标志 1: 相等 0: 不等
NPC[31:0]	O	32位输出

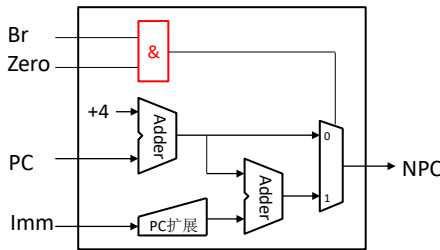
计算机组成与实现

Beq建模

本质上，beq涉及2大功能

- 功能1：根据比较的结果，计算PC。这属于NPC的功能范畴
- 功能2：寄存器比较。让ALU执行减法，然后把比较结果zero传递给NPC

对于NPC，现在需要知道当前指令是否是beq及zero的结果



Br	Zero	MUX
0	0	0
0	1	0
1	0	0
1	1	1

```
if ( R[rs] == R[rt] )
    PC ← PC+4 + sign_ext(imm16||00)
else
    PC ← PC+4
```

信号名	方向	描述
PC[31:0]	I	32位输入
Imm[15:0]	I	16位立即数
Br	I	beq指令标志 1: 当前指令是beq 0: 当前指令不是beq
Zero	1	rs和rt相等标志 1: 相等 0: 不等
NPC[31:0]	O	32位输出

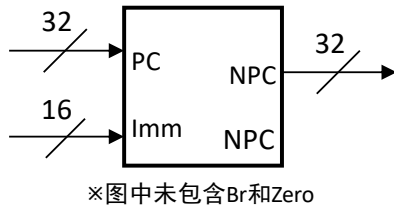
计算机组成与实现

Beq建模

本质上，beq涉及2大功能

- 功能1：根据比较的结果，计算PC。这属于NPC的功能范畴
- 功能2：寄存器比较。让ALU执行减法，然后把比较结果zero传递给NPC

对于NPC，现在需要知道当前指令是否是beq及zero的结果



```
if ( R[rs] == R[rt] )
    PC←PC+4 + sign_ext(imm16||00)
else
    PC←PC+4
```

信号名	方向	描述
PC[31:0]	I	32位输入
Imm[15:0]	I	16位立即数
Br	I	beq指令标志 1: 当前指令是beq 0: 当前指令不是beq
Zero	1	rs和rt相等标志 1: 相等 0: 不等
NPC[31:0]	O	32位输出

计算机组成与实现

Beq建模

NPC计算PC

ALU执行减法

```
if ( R[rs] == R[rt] )
    PC←PC+4 + sign_ext(imm16||00)
else
    PC←PC+4
```

环节	步骤	语义	连接关系	功能部件	控制信号取值
1	读指令	读取指令 计算下条指令地址 更新PC	<PC.DO, IM.A> <PC.DO, NPC.PC> <IM.D[15:0], NPC.Imm> <NPC.NPC, PC.DI>	IM PC NPC	Br:1 Zero:ALU的 zero
2	读操作数		<IM.D[25:21], RF.A1> <IM.D[20:16], RF.A2 >	RF	
3	执行	ALU执行减法	<RF.RD1, ALU.A> <RF.RD2, ALU.B>	ALU	ALUOp:SUB

指令	NPC		PC	IM	RF				ALU		DM	
	PC	Imm	NPC	A	A1	A2	A3	WD	A	B	A	WD
beq	PC.PC	IM.D[15:0]	NPC.NPC	PC.PC	IM.D[25:21]	IM.D[20:16]			RF.RD1	RF.RD2		

计算机组成与实现

及时调整已有设计

- 一旦增加了新的功能部件或修改了已有的功能部件，就需要及时调整已经完成的设计
- 示例：由于修改了NPC的设计，因此需要调整addu的设计
 - 在beq之前的设计中，由于各指令均顺序执行，因此NPC是单功能部件
 - 现为满足beq的计算需求调整了NPC的设计，所以需修改addu指令的设计
 - subu、lw、sw等指令类似

环节	步骤	语义	连接关系	功能部件	控制信号取值
1	读指令	读取指令 计算下条指令地址 更新PC	<PC.DO,IM.A> <PC.DO,NPC.PC> <NPC.NPC,PC.DI>	IM PC NPC	Br:0
2	读操作数		<IM.D[25:21],RF.A1> <IM.D[20:16],RF.A2>	RF	
3	执行	ALU执行加法	<RF.RD1,ALU.A> <RF.RD2,ALU.B>	ALU	ALUOp:ADD
4	回写	计算结果回写至rd寄存器	<ALU.C,RF.WD> <IM.D[15:11],RF.A3>	RF	RFWr:1

※后续还有类似的问题

计算机组成与实现

目录

- 设计方法学概述
- 单条指令的数据通路的建模方法
 - Jal
- 完整数据通路的综合方法

计算机组成与实现

Jal建模

$$PC \leftarrow PC_{31..28} \parallel instr_index \parallel 0^2$$
$$R[31] \leftarrow PC+4$$

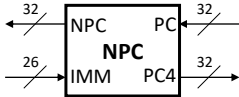
包含2个操作：①计算PC值；②将PC+4写入R31

NPC改造1：支持新的PC计算需求

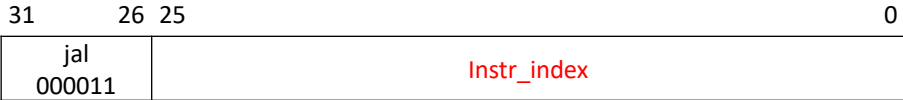
- NPC的功能定位是计算PC值
- NPC已输入指令低16位，只需要扩大至指令低26位，就能满足全部需求
- NPC的计算需求包含3种情况，分别是PC+4、PC+4+偏移、jal地址；必须将br调整为NPCOp[1:0]
 - br信号只有1位，不能表达3种含义

NPC改造2：支持回写

- NPC已经计算出了PC+4，因此只需要将PC+4输出即可



※图中未包含控制类信号



计算机组成与实现

Jal建模

$$PC \leftarrow PC_{31..28} \parallel instr_index \parallel 0^2$$
$$R[31] \leftarrow PC+4$$

调整后的NPC的I/O信号

信号名	方向	描述	调整说明
PC[31:0]	I	32位输入	
Imm[25:0]	I	26位立即数	从Imm[15:0]调整为Imm[25:0]
NPCOp[1:0]	I	NPC功能选择 00：计算顺序地址（PC+4） 01：计算beq地址 10：计算jal地址 11：保留	取消1位的Br，改为2位的NPCOp
Zero	I	rs和rt相等标志 1：相等 0：不等	
NPC[31:0]	O	32位输出	
PC4[31:0]	O	32位输出	输出PC+4值

注意：由于NPC的控制信号发生了变化，因此前面各条指令的NPC控制信号取值也需要随之调整。

计算机组成与实现

Jal建模

$$PC \leftarrow PC_{31..28} \parallel instr_index \parallel 0^2$$
$$R[31] \leftarrow PC + 4$$

- 对于回写数据来说，数据来自NPC输出的PC4
- 对于回写寄存器编号来说，需要直接表示为0x1F
 - 其并未出现在指令格式中，而是固定为31

环节	步骤	语义	连接关系	功能部件	控制信号取值
1	读指令	读取指令 计算下条指令地址 更新PC	$\langle PC.DO, IM.A \rangle$ $\langle PC.DO, NPC.PC \rangle$ $\langle IM.D[25:0], NPC.Imm \rangle$ $\langle NPC.NPC, PC.DI \rangle$	IM PC NPC	$NPCOp: JAL$
2	回写		$\langle 0x1F, RF.A3 \rangle$ $\langle NPC.PC4, RF.WD \rangle$	RF	$RFWr: 1$

指令	NPC		PC	IM	RF				EXT	ALU		DM	
	PC	Imm	DI	A	A1	A2	A3	WD	Imm	A	B	A	WD
jal	PC.DO	IM.D[25:0]	NPC.NPC	PC.DO			0x1F	NPC.PC4					

计算机组成与实现

目录

- 设计方法学概述
- 单条指令的数据通路的建模方法
 - Jr
- 完整数据通路的综合方法

计算机组成与实现

Jr建模

$PC \leftarrow R[rs]$

- 设计分歧：虽然Jr的功能很简单，但却有2种可能思路
 - 方案1：将RF输出的rs值直接输出至PC
 - 方案2：将RF输出的rs值先输出至NPC，然后再从NPC输出至PC
- 基本思路：遵循某些基本原则或方法
- 基本原则：“高内聚低耦合”中的最少知识原则
 - 方案1：PC与NPC都是“知道”如何计算PC值的功能部件
 - 方案2：NPC是唯一“知道”如何计算PC值的功能部件

31	26	25	21	20	11	10	6	5	0
special		rs		0		0		jr	
000000				00 0000 0000		00000		001000	

计算机组成与实现

Jr建模

$PC \leftarrow R[rs]$

- NPC接口：①增加函数返回地址RA[31:0]；②增加新的计算模式

信号名	方向	描述	调整说明
PC[31:0]	I	32位输入	
Imm[25:0]	I	26位立即数	从Imm[15:0]调整为Imm[25:0]
RA[31:0]	I	32位返回地址	rs寄存器保存的返回地址
NPCOp[1:0]	I	NPC功能选择 00：计算顺序地址（PC+4） 01：计算beq地址 10：计算jal地址 11：计算jr地址	NPCOp的0b11项用于产生jr相关的PC目的地址
Zero	I	rs和rt相等标志 1：相等 0：不等	
NPC[31:0]	O	32位输出	
PC4[31:0]	O	32位输出	输出PC+4值

注意：由于NPC的控制信号发生了变化，因此前面各条指令的NPC控制信号取值也需要随之调整。

计算机组成与实现

Jr建模

PC←R[rs]

推理发现存在冲突：同一个部件执行多个操作。如何选择？

- 在环节1和3，NPC执行不同的计算功能

环节	步骤	语义	连接关系	功能部件	控制信号取值
1	读指令	读取指令 计算下条指令地址 更新PC	<PC.DO, IM.A> <PC.DO, NPC.PC> <NPC.NPC, PC.DI>	IM PC NPC	NPCOp:PC+4 ①
2	读操作数		<IM.D[26:21], RF.A1>	RF	
3	计算返回地址		<RF.RD1, NPC.RA>	NPC	NPCOp:JR ②
4	更新PC	将rs值写入PC	<NPC.NPC, PC.DI>	PC	

选择后执行的，即环节3。原因：

- 单周期特点是所有操作都必须在一个时钟周期内完成
- 逻辑上，后发生的事件是会覆盖先发生的事件的

```
// a的值为6
a = 5 ;
a = 6 ;
```

计算机组成与实现

Jr建模

PC←R[rs]

单周期的特点是所有操作都必须在一个时钟周期内完成。这意味

环节	步骤	语义	连接关系	功能部件	控制信号取值
1	读指令	读取指令 计算下条指令地址 更新PC	<PC.DO, IM.A> <PC.DO, NPC.PC> <NPC.NPC, PC.DI>	IM PC NPC	NPCOp:PC+4 ①
2	读操作数		<IM.D[26:21], RF.A1>	RF	
3	计算返回地址		<RF.RD1, NPC.RA>	NPC	NPCOp:JR ②
4	更新PC	将rs值写入PC	<NPC.NPC, PC.DI>	PC	

指令	NPC			PC	IM	RF				EXT	ALU		DM	
	PC	Imm	RA	DI	A	A1	A2	A3	WD	Imm	A	B	A	WD
jal	PC.DO		RF.RD1	NPC.NPC	PC.DO	IM.D[25:21]								

计算机组成与实现

目录

- 设计方法学概述
- 单条指令的数据通路的建模方法
- 完整数据通路的综合方法

计算机组成与实现

汇聚

- 将独立建模的指令级别数据通路汇聚在一起

部件	PC	NPC			IM	RF				EXT	ALU		DM
输入信号	DI	PC	Imm	RA	A	A1	A2	A3	WD		A	B	A
addu	NPC.NPC	PC.DO			PC.DO	IM.D[25:21]	IM.D[20:16]	IM.D[15:11]	ALU.C		RF.RD1	RF.RD2	
subu	NPC.NPC	PC.DO			PC.DO	IM.D[25:21]	IM.D[20:16]	IM.D[15:11]	ALU.C		RF.RD1	RF.RD2	
ori	NPC.NPC	PC.DO			PC.DO	IM.D[25:21]		IM.D[20:16]	ALU.C	IM.D[15:0]	RF.RD1	EXT.Ext	
lw	NPC.NPC	PC.DO			PC.DO	IM.D[25:21]		IM.D[20:16]	DM.RD	IM.D[15:0]	RF.RD1	EXT.Ext	ALU.C
sw	NPC.NPC	PC.DO			PC.DO	IM.D[25:21]				IM.D[15:0]	RF.RD1	EXT.Ext	ALU.C
beq	NPC.NPC	PC.DO	IM.D[15:0]		PC.DO	IM.D[25:21]	IM.D[20:16]				RF.RD1	RF.RD2	
jal	NPC.NPC	PC.DO	IM.D[25:0]	RF.RD1	PC.DO			0x1F	NPC.PC4				
jr	NPC.NPC	PC.DO			PC.DO	IM.D[25:21]							

计算机组成与实现

综合

□ 归并每个输入信号的信号来源：保留不同来源

部件	PC	NPC			IM	RF				EXT	ALU		DM
输入信号	DI	PC	Imm	RA	A	A1	A2	A3	WD		A	B	A
addu	NPC.NPC	PC.DO			PC.DO	IM.D[25:21]	IM.D[20:16]	IM.D[15:11]	ALU.C		RF.RD1	RF.RD2	
subu	NPC.NPC	PC.DO			PC.DO	IM.D[25:21]	IM.D[20:16]	IM.D[15:11]	ALU.C		RF.RD1	RF.RD2	
ori	NPC.NPC	PC.DO			PC.DO	IM.D[25:21]		IM.D[20:16]	ALU.C	IM.D[15:0]	RF.RD1	EXT.Ext	
lw	NPC.NPC	PC.DO			PC.DO	IM.D[25:21]		IM.D[20:16]	DM.RD	IM.D[15:0]	RF.RD1	EXT.Ext	ALU.C
sw	NPC.NPC	PC.DO			PC.DO	IM.D[25:21]				IM.D[15:0]	RF.RD1	EXT.Ext	ALU.C
beq	NPC.NPC	PC.DO	IM.D[15:0]		PC.DO	IM.D[25:21]	IM.D[20:16]				RF.RD1	RF.RD2	
jal	NPC.NPC	PC.DO	IM.D[25:0]	RF.RD1	PC.DO			0x1F	NPC.PC4				
jr	NPC.NPC	PC.DO			PC.DO	IM.D[25:21]							
完整	NPC.NPC	PC.DO	IM.D[25:0] ×	RF.RD1	PC.DO	IM.D[25:21]	IM.D[20:16]	IM.D[15:11] IM.D[20:16] 0x1F	ALU.C DM.RD NPC.PC4	IM.D[15:0]	RF.RD1	RF.RD2 EXT.Ext	ALU.C

※由于IM.D[25:0]覆盖了IM.D[15:0]，因此只保留前者即可

计算机组成与实现

构造MUX

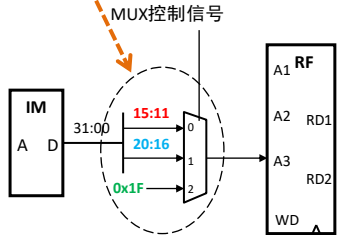
□ 以RF.A3为例，综合后的结果表明该信号有3个输入源：

◆ IM.D[15:11]、IM.D[20:16]、0x1F

部件	PC	NPC			IM	RF				EXT	ALU		DM
输入信号	DI	PC	Imm	RA	A	A1	A2	A3	WD		A	B	A
完整	NPC.NPC	PC.DO	IM.D[25:0]	RF.RD1	PC.DO	IM.D[25:21]	IM.D[20:16]	IM.D[15:11] IM.D[20:16] 0x1F	ALU.C DM.RD NPC.PC4	IM.D[15:0]	RF.RD1	RF.RD2 EXT.Ext	ALU.C

□ 根据数字电路知识可知，必须部署一个MUX

□ 由此可见，MUX是“自动”综合出来



※在控制部分再详述MUX的工程化规范设计

计算机组成与实现

构造MUX

□ MUX各路输入信号分派并无特定的原则

◆ 示例：RF的A3信号

指令与RF.A3	
指令	RF.A3
addu	IM.D[15:11]
subu	IM.D[15:11]
ori	IM.D[20:16]
lw	IM.D[20:16]
sw	
beq	
jal	0x1F
jr	

RF.A3的MUX	
端口编号	输入源
0	IM.D[15:11]
1	IM.D[20:16]
2	0x1F



注意

MUX端口与输入信号的对应关系是任意对应的
但是一旦确定对应关系后，则必须固定下来

计算机组成与实现

数据通路的两种表示方式

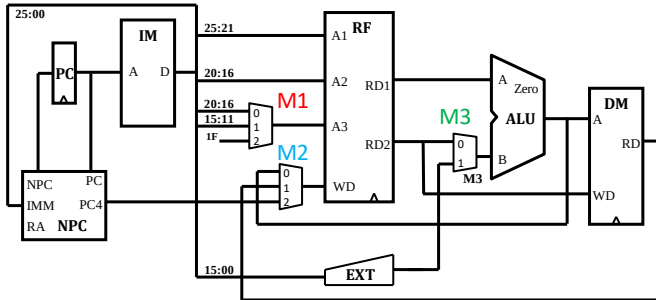
□ 从形式建模的数据通路可以很容易的构造出图形化的数据通路

部件	PC	NPC			IM	RF				EXT	ALU		DM
输入信号	DI	PC	Imm	RA	A	A1	A2	A3	WD		A	B	A
完整	NPC.NPC	PC.DO	IM.D[25:0]	RF.RD1	PC.DO	IM.D[25:21]	IM.D[20:16]	IM.D[15:11] IM.D[20:16] 0x1F	ALU.C DM.RD NPC.PC4	IM.D[15:0]	RF.RD1	RF.RD2 EXT.Ext	ALU.C

M1

M2

M3



计算机组成与实现

从设计模型到VerilogHDL

□ 示例：ALU的B输入

部件	PC	NPC			IM	RF				EXT	ALU		DM
输入信号	DI	PC	Imm	RA	A	A1	A2	A3	WD	Imm	A	B	A
完整	NPC.NPC	PC.DO	IM.D[25:0]	RF.RD1	PC.DO	IM.D[25:21]	IM.D[20:16]	IM.D[15:11] IM.D[20:16] 0x1F	ALU.C DM.RD NPC.PC4	IM.D[15:0]	RF.RD1	RF.RD2 EXT.Ext	ALU.C

```
wire      [31:0]      RD2 ;
wire      [31:0]      Ext ;
wire      [31:0]      ALU_B ;

RF        U_RF( ..., RD2, ... ) ;           // 实例化寄存器堆
EXT       U_EXT( ..., Ext, ... ) ;          // 实例化扩展单元
ALU       U_ALU( ..., ALU_B, ... ) ;        // 实例化ALU

// 实例化
MUX32_2_1 U_MUX_ALUB( RD2, Ext, ALU_B, ALUBSrc ) ;
```